

Loops and Functions (Answers)

Learning Outcomes

In this Session of our R Workshop, we will cover the following introductory aspects of R Studio & Programming:

- Functions, including
 - built-in
 - library
 - user-defined
- File Input and Output
- Getting help
- Conditional and Flow Control Statements
 - if
 - for
 - while

By the end of this Session, you will understand:

1. The three types of functions
2. The anatomy of a function
3. How to install and use R Libraries
4. How to use documentation, get help in RStudio
5. How to import and save data
6. How to write an if statement and what to use it for
7. How to write a for loop, how it works and what to use if for
8. How to write a while loop, how it works and what to use it for

Functions

We will look at three types of functions:

- Built-in
 - are available when R loads without further info
- External package functions
 - need to be initially installed
 - when you want to use them you must load them using the `library()` command
- User-defined functions
 - Your babies. You make these to:
 - * perform discrete tasks
 - * automate repetitive tasks
 - * reduce errors
 - * make code more readable
 - * organize code

We'll talk about each of these and explore some examples. The following vocabulary will be helpful:

define: before a function can be used it must be defined. Built-in and external package functions are defined in the packages associated with them. When you create your own functions (user-defined) you will create a definition.

call: when you use a function in your program you “call” it. You can't call a function before it is defined.

A function has three main components:

- name (so you can refer to it because “hey, you!” won't get its attention)
- parameters
 - pieces of information the function needs to do it's job.
 - parameters can be required or optional
 - * the function won't run without required parameters
 - * optional parameters will have a default value which is used if no value is passed for it
 - * the term parameters refers to the variables the function needs to do its job.
 - * when we provide values for those parameters, we call them arguments.
- return value
 - this is a value that the function returns when it's done

- frequently we will store this value in a variable using the assignment operator
- the return value may be
 - * a value that was calculated
 - * data that has been manipulated in some way, or
 - * a status that indicates whether the function encountered any errors

Built-In Functions

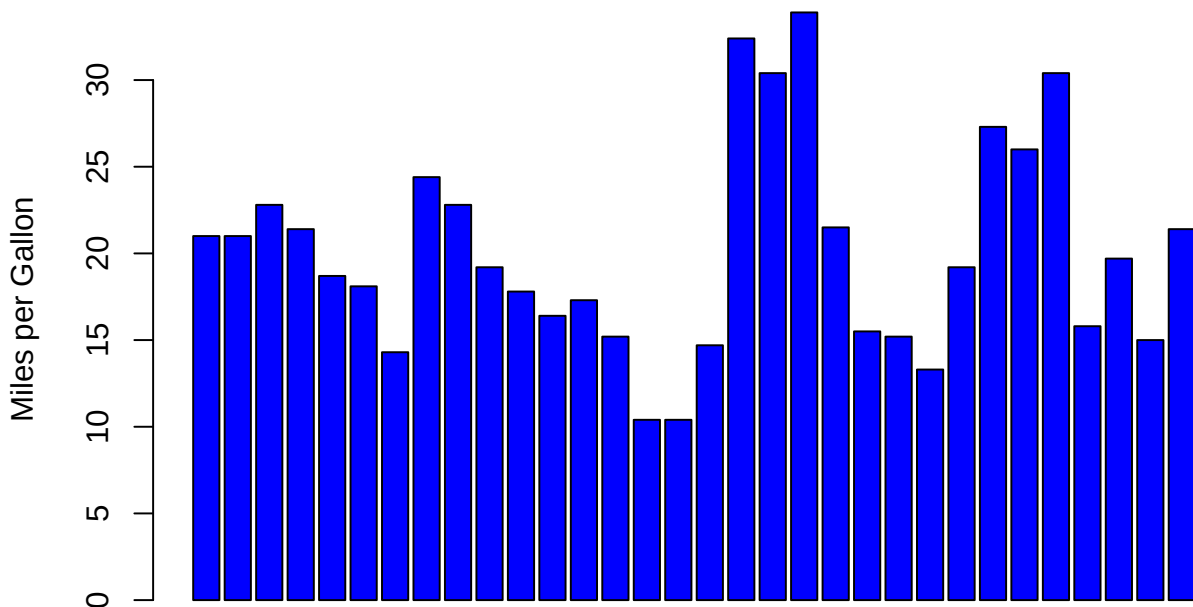
Built-in functions are functions that are loaded when R is loaded. You don't have to do anything to use them.

You can see what packages R loads by opening the Packages tab in the Output pane. If you haven't loaded any other packages, then the packages with check-marks are the built-in packages that get loaded.

We saw an example of using built-in package in the Basic Syntax and Operators Workshop.

```
# define some data
data <- mtcars

# generate plot
bp <- barplot(data$mpg,
               names.arg = data$Carb,
               col="blue",
               ylab="Miles per Gallon")
```



```
bp
```

```
      [,1]
 [1,]  0.7
 [2,]  1.9
 [3,]  3.1
 [4,]  4.3
 [5,]  5.5
 [6,]  6.7
 [7,]  7.9
 [8,]  9.1
 [9,] 10.3
[10,] 11.5
[11,] 12.7
[12,] 13.9
[13,] 15.1
[14,] 16.3
[15,] 17.5
[16,] 18.7
[17,] 19.9
[18,] 21.1
[19,] 22.3
[20,] 23.5
[21,] 24.7
[22,] 25.9
[23,] 27.1
[24,] 28.3
[25,] 29.5
[26,] 30.7
[27,] 31.9
[28,] 33.1
[29,] 34.3
[30,] 35.5
[31,] 36.7
[32,] 37.9
```

Want more information about a function? Use `?<function name>`.

The `barplot()` function is part of Base R and is loaded by default. In the console window, type `“?barplot”` and hit enter. Let’s use the vocabulary we covered earlier and read some documentation!

Here is what we see in the Help pane:

- name of the function and the package it is in: `barplot {package}`
- title of the help page: Bar Plots
- Description: gives a high-level description of what the function does
- Usage:
 - provides the function interface
 - * all parameters, required and optional
 - optional parameters will be followed by an “=” sign
 - * different “flavors” of the function
- Arguments
 - lists all parameters
 - sometimes indicates “optional”, but not always
 - includes acceptable values, data types and data structures
- Value
 - provides information about the return value
- Authors, References, See Also and Examples
 - self-explanatory
 - click on “Run examples”

Practice Using Built-In Functions

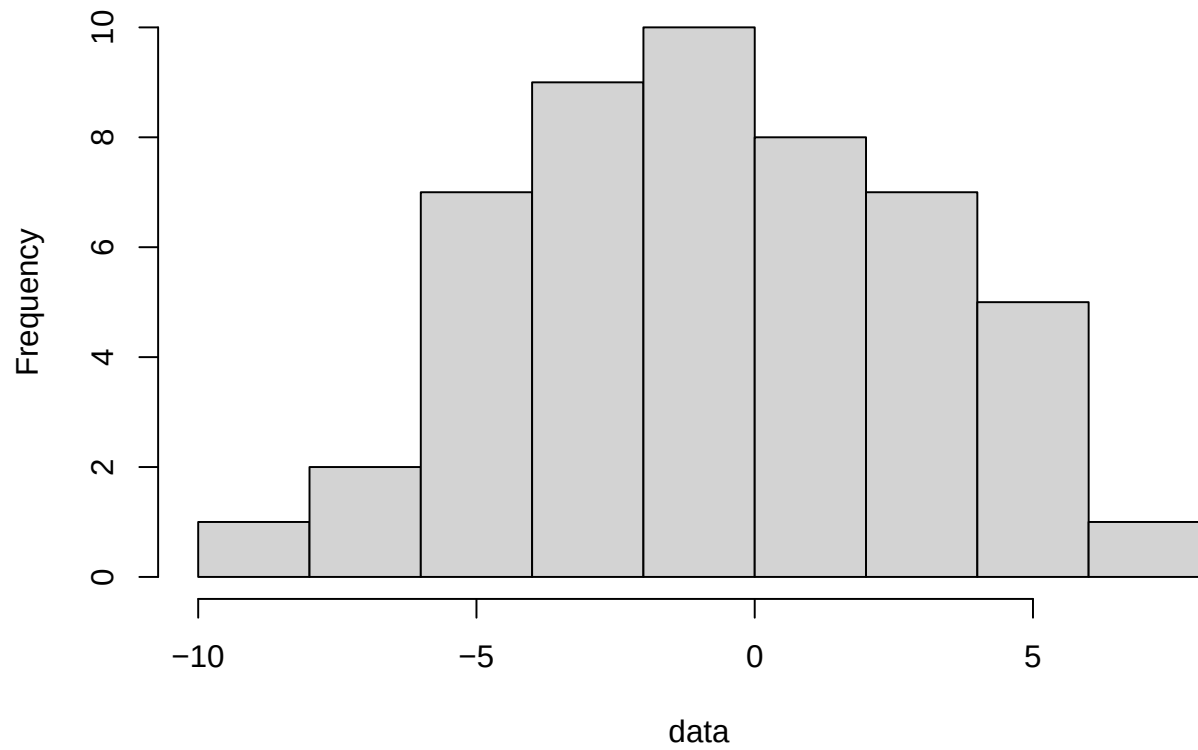
```
# You can use the function built-in function hist() to
# create a histogram of data

# Use ?hist() to learn how to call the function
?hist()

# Generate 50 values from a normal distribution with mean=0 and standard
# deviation=4.25
data <- rnorm(50, mean = 0, sd = 4.25)

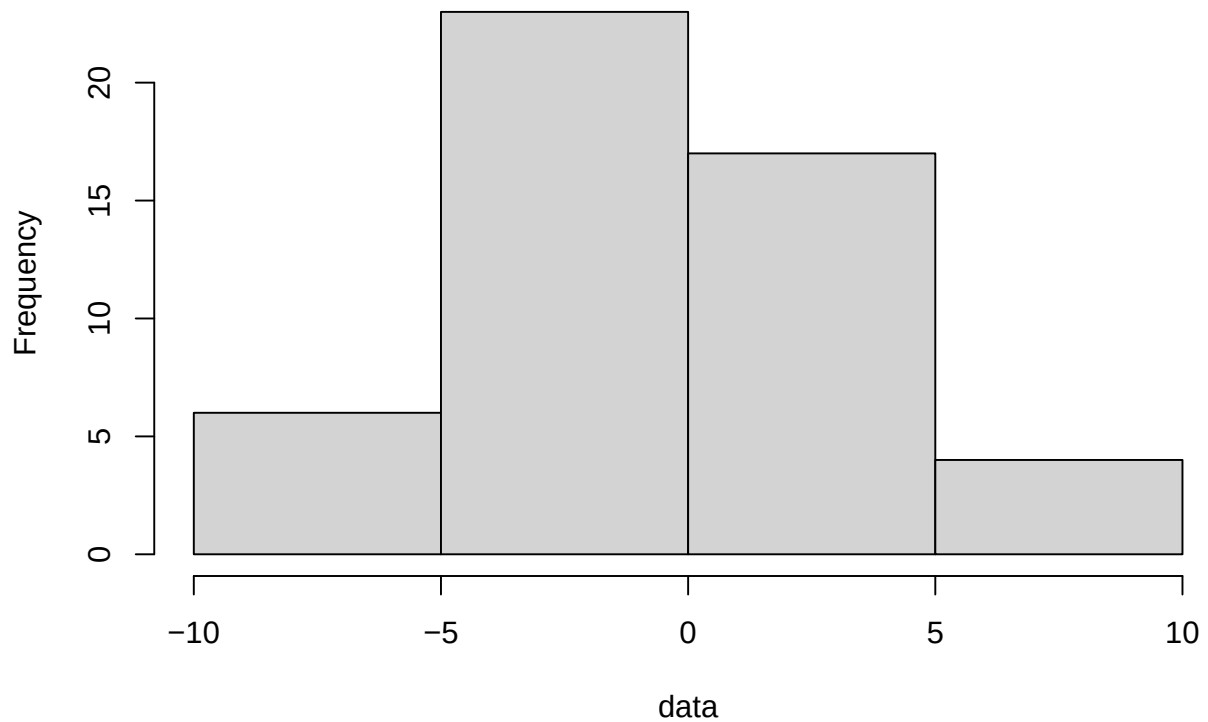
# Call the function, pass only the data
hist(data)
```

Histogram of data



```
# add a breaks statement to increase the number of bars (bins)
# hint: count the bars in the last plot and increase that number
hist(data, breaks=3)
```

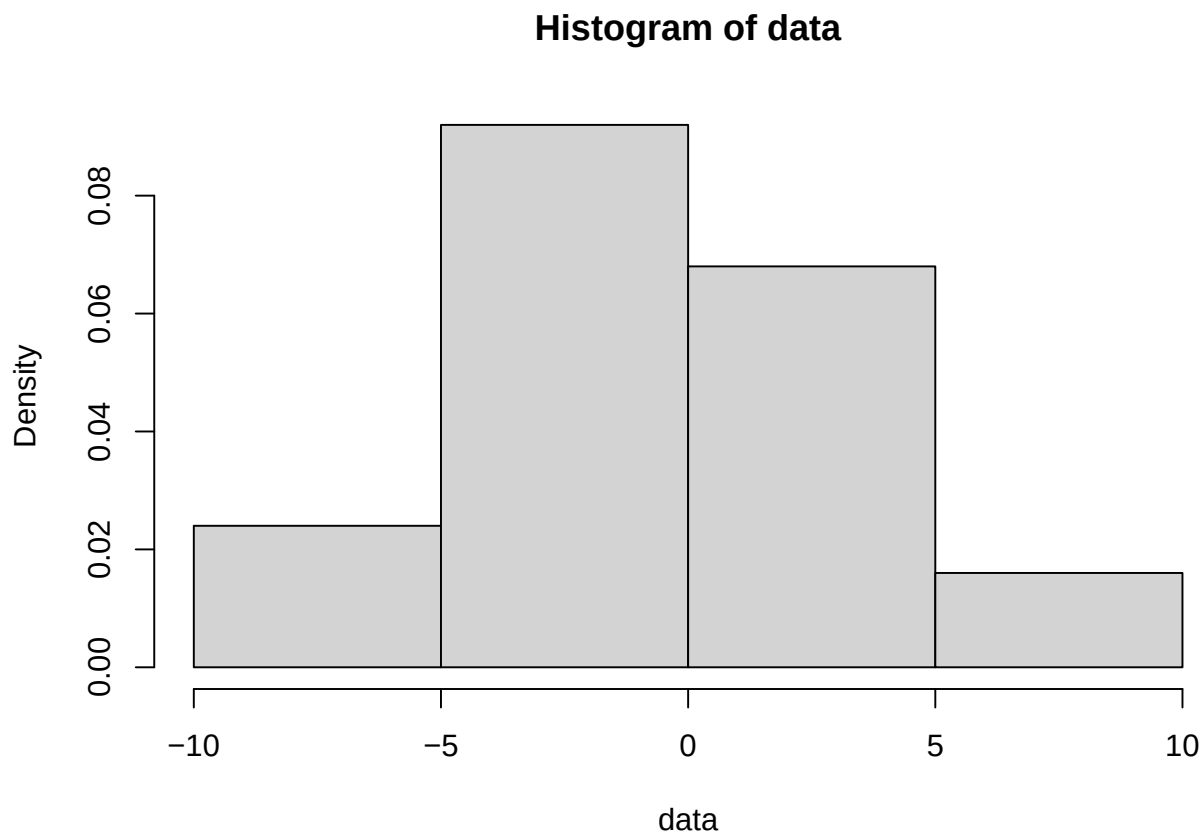
Histogram of data



```
# NOTE: behind the scenes, R uses `pretty()` which will adjust your bin edges so # that they
# number you asked for!

# set the freq argument to TRUE and then FALSE
# look at the difference
hist(data, breaks=3, freq = TRUE) # this is the default behavior

# set freq argument to FALSE
hist(data, breaks=3, freq = FALSE)
```



Packages

The Base R packages provide a lot of functionality and they will take you far. However, the beauty of open source languages is that creative, capable people like you can create packages and provide them to you. The ability to expand R's functionality is limitless and free.

Packages:

- must be installed initially
- once installed, they must be loaded using the `library()` function
 - the function will stay loaded as long as the R-session is active and not closed.
 - if the R session is closed (you exit out of R), you will need to run the `library()` command again

Let's explore a package that will be useful for visualizing data: `ggplot2`.

First, let's see what packages are installed.

```
print(installed.packages())
```

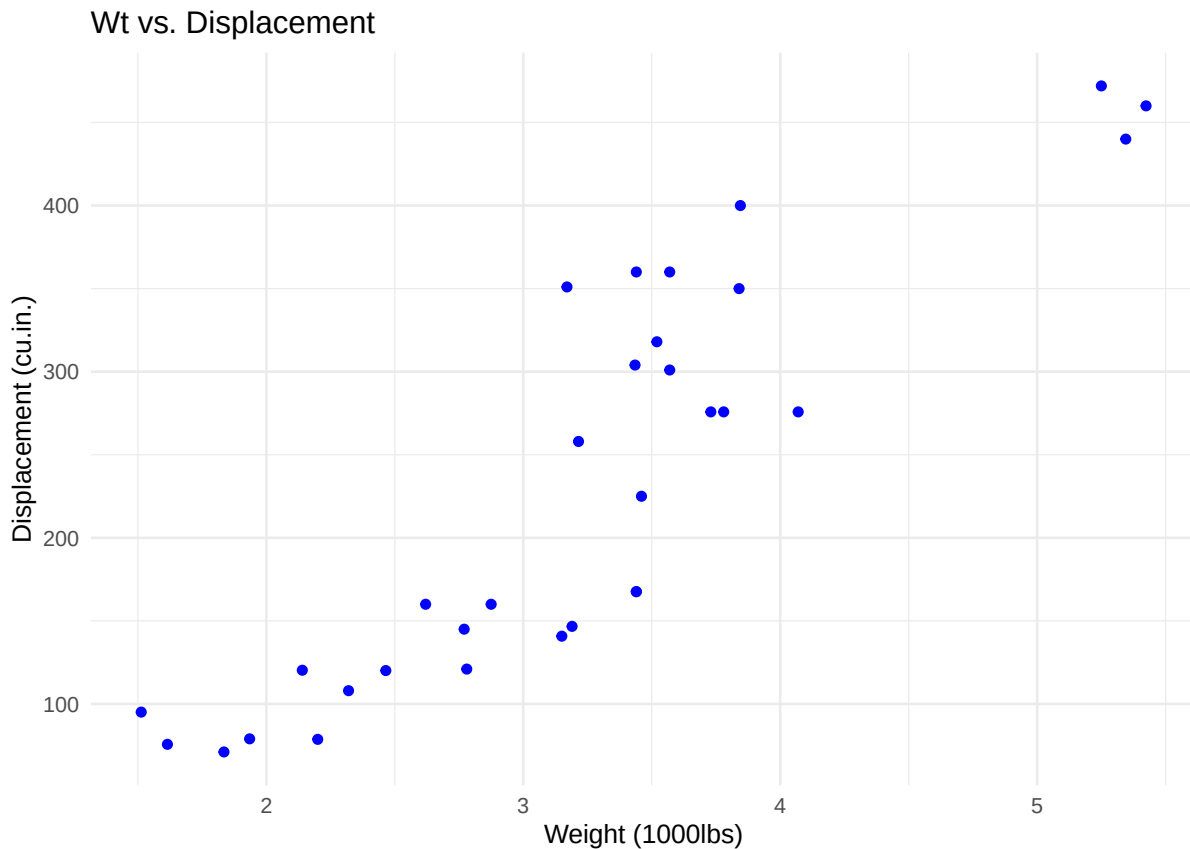
This let's us see what packages pre-installed. If our package isn't installed, we need to install it. Let's install `ggplot2`.

```
#install.packages("ggplot2")
```

Once a package is installed, we can load it whenever we want to use it. When do we need to load a library? Each time we start R, we start an R Session. We need to load packages every time we start a new session. If you select **Session/New Session** from the menu, you need to reload your packages in the new session. We load a library using the `library()` function.

```
# load library
library(ggplot2)

# use library
ggplot(data=mtcars, aes(x=wt, y=disp)) +
  geom_point(col="blue") +
  labs(title="Wt vs. Displacement",
       x = "Weight (1000lbs)",
       y = "Displacement (cu.in.)") +
  theme_minimal()
```



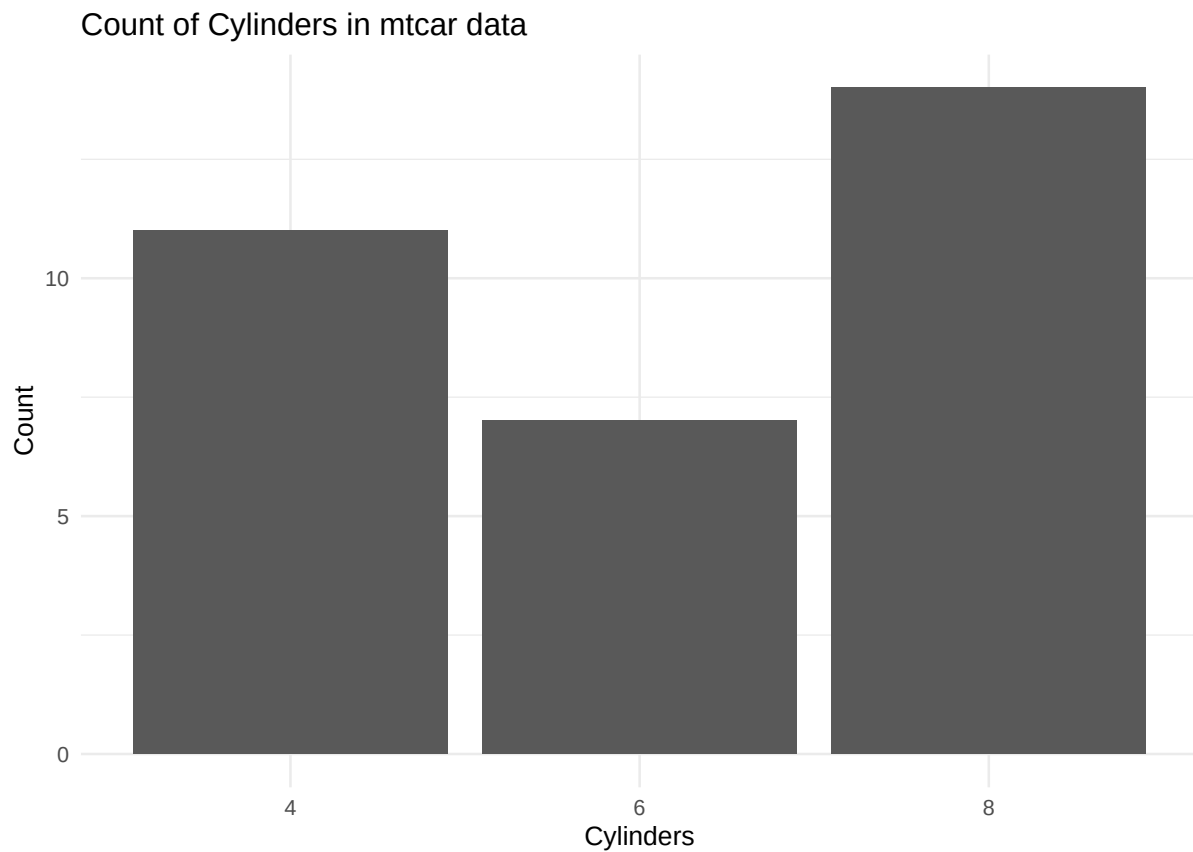
We can use the `require()` function to check if a package is installed. If it isn't, we can install it and if it is we can just load it. `require()` returns true if the package IS installed. We want to install it if it isn't. The `if()` function below executes the code if the expression between the outer parentheses evaluates to `TRUE`. For this reason, we use the expression `!require(ggplot2)`.

```
# If ggplot2 package isn't installed, install it
# if (!require("ggplot2")) {
#   install.packages("ggplot2")
# }

# load library
library("ggplot2")

# use the library
ggplot(mtcars, aes(x=as.factor(cyl))) +
  geom_bar(stat="count") +
  labs(title = "Count of Cylinders in mtcars data",
       x = "Cylinders",
```

```
y = "Count") +  
theme_minimal()
```



User-defined Functions

Functions should:

- perform a discrete task or set of tasks
- be written to be reusable

Whether you are just starting out or a seasoned programmer, spending time with paper and pencil before programming is a good idea. Consider:

- what will the function do?
- what information (parameters) does it need to do it
- what parameters will be required vs. optional
 - it's ok to make everything required

- what will the function return?

```
# A function that calculates a library fine
calculate_fine <- function(days_overdue, rate_per_day = 0.25) {
  fine <- days_overdue * rate_per_day
  return(fine)
}

# Calling it with values
calculate_fine(4)          # 4 days overdue at the default rate -> 1
```

```
[1] 1
```

```
calculate_fine(10, 0.50)   # 10 days overdue at 50 cents/day -> 5
```

```
[1] 5
```

```
# calling it with variables
days_overdue <- 7
rate_per_day <- .20
calculate_fine(days_overdue, rate_per_day)
```

```
[1] 1.4
```

Practice Writing a User-Defined Function

Your mission, if you choose to accept it (accept it! accept it!) is to write a function that converts a temperature from Celsius to Fahrenheit. Recall the conversion formula (where C represents degrees Celsius and F represents degrees Fahrenheit):

$$F = \frac{9}{5}C + 32 \quad (1)$$

To get started, define: 1. What info do you need to provide the function? This will define your parameters. 2. What info do you want to get back from the function? This will define your output (return value). 3. What do you need to transfer the input (e.g. parameters) into the output (e.g. return value)? This will define the code steps you need in your function.

```

celsius2fahr <- function(temp.celsius) {
  fahrenheit <- (9/5)*temp.celsius + 32
  return(fahrenheit)
}

# test our code
temp.celsius <- 35
temp.fahr <- celsius2fahr(temp.celsius)
print(temp.fahr)

```

```
[1] 95
```

You now know what built-in, package and user-defined functions are. You've used help to get more information about using built-in and package functions and you've written a simple function on your own.

Next, we will look at conditional and control flow statements that will expand what you can do with your code, including your user-defined functions!

Conditional Statements and Flow Control

IF Statements (Conditional Flow)

The `if` statement:

- runs code based on whether a condition is met
- if condition is met, code will be run once

The expression you use for your condition must evaluate to `TRUE` or `FALSE`. Examples of these types of expressions include:

- boolean/logical variables, e.g. `runIF`
- relational expressions e.g. `mynumber > 5`
- logical expressions e.g. `isHelpful && isAvailable`

```

# Logical variable
runIF = TRUE

if (runIF) {
  print("if condition met")
}

```

```
[1] "if condition met"
```

```
# relational expression
val1 = 3

if (val1 < 5) {
  print("Number is less than 5.")
}
```

```
[1] "Number is less than 5."
```

```
# logical operators

if (val1 > 1 && val1 < 5) {
  print("Number is between 1 and 5 (not inclusive)")
}
```

```
[1] "Number is between 1 and 5 (not inclusive)"
```

Sometimes, you want to do something **else** if the **if** condition isn't met. In these cases you can use add an **else** statement.

```
if (runIF) {
  print("if condition is met.")
} else {
  print("if condition is not met.")
}
```

```
[1] "if condition is met."
```

```
# You can use more than one else statement
val1 <- 3

if (val1==1) {
  print('Value = 1')
} else if (val1==2) {
  print('Value = 2')
} else {
  print('Value = 3') # assumes val1 will only have a value of 1, 2 or 3
}
```

```
[1] "Value = 3"
```

NOTES:

1. It is important that you have open '{' and closed '}' braces for each part of your if statements (e.g. `if`, `else if`, and `else`)
2. You can check closure of your braces by putting your cursor next to a brace. This puts a little white box around the brace your cursor is next to as well as around the matching brace.
3. If you type {, R produces the closed brace.

FOR Loops

A for loop:

- runs a specific number of times
- will run at least once
- is useful for traversing data structures like vectors, matrices, and dataframes.

```
# basic
for (i in 1:5) {
  print(i)
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

```
# iterate over a vector
vec1 <- c(2, 4, 6, 8, 10)

for (n in vec1) {
  print(n)
}
```

```
[1] 2
[1] 4
[1] 6
[1] 8
[1] 10
```

```
# nested for loop
for (i in 1:5) {
  print(paste("i=",i))
  for (j in 1:3) {
    print(paste("  j=",j))
  }
}
```

```
[1] "i= 1"
[1] "  j= 1"
[1] "  j= 2"
[1] "  j= 3"
[1] "i= 2"
[1] "  j= 1"
[1] "  j= 2"
[1] "  j= 3"
[1] "i= 3"
[1] "  j= 1"
[1] "  j= 2"
[1] "  j= 3"
[1] "i= 4"
[1] "  j= 1"
[1] "  j= 2"
[1] "  j= 3"
[1] "i= 5"
[1] "  j= 1"
[1] "  j= 2"
[1] "  j= 3"
```

A nested `for` loop is useful for accessing data in a matrix. **NOTE:** this takes some thought regarding what order you want to access elements in the matrix.

```
# create some data
data <- sample(1:50, 35, replace=TRUE)
# Put data in matrix form
matrix.2D <- matrix(data=data, nrow=5, ncol=7, byrow=FALSE)

for (i in 1:nrow(matrix.2D)) {
  for (j in 1:ncol(matrix.2D)) {
    print(matrix.2D[i,j])
  }
}
```

```
[1] 20
[1] 26
[1] 30
[1] 43
[1] 38
[1] 35
[1] 42
[1] 26
[1] 2
[1] 26
[1] 35
[1] 39
[1] 45
[1] 44
[1] 34
[1] 15
[1] 11
[1] 8
[1] 20
[1] 41
[1] 35
[1] 35
[1] 11
[1] 46
[1] 48
[1] 36
[1] 15
[1] 1
[1] 34
[1] 17
[1] 2
[1] 49
[1] 5
[1] 34
[1] 12
```

The code above prints each row column by column. What if you wanted to print each column row by row? What changes would you need to make? Can you just swap `i` and `j`?

```
# create some data
data <- sample(1:50, 35, replace=TRUE)
# Put data in matrix form
```

```

matrix.2D <- matrix(data=data, nrow=5, ncol=7, byrow=FALSE)

# copy the for loop above and make changes to it so values are printed out
# column by column

for (i in 1:ncol(matrix.2D)) {
  for (j in 1:nrow(matrix.2D)) {
    print(matrix.2D[j,i])
  }
}

```

```

[1] 35
[1] 25
[1] 44
[1] 28
[1] 42
[1] 11
[1] 39
[1] 39
[1] 42
[1] 46
[1] 31
[1] 39
[1] 14
[1] 16
[1] 3
[1] 19
[1] 43
[1] 12
[1] 19
[1] 16
[1] 40
[1] 12
[1] 49
[1] 33
[1] 22
[1] 15
[1] 45
[1] 14
[1] 45
[1] 28
[1] 3

```

```
[1] 43
[1] 26
[1] 39
[1] 45
```

WHILE Loops

A while loop:

- executes code by testing a condition
- code may execute once, several times or not at all
- can be used to run code in response to a changing condition
- the condition can be a relational expression or logical expression
- the condition can be a logical variable, however, we'll see that this can be inefficient

NOTES:

- the variables associated with the condition being tested must be set before the while loop runs
- the condition must be updated within the while loop

The example below uses a relational expression (with relational operator) to “drive” the while loop.

```
# Using a relational expression
val1 <- 5 # IMPORTANT STATEMENT # 1

while (val1 < 10) {
  # The expression between the parentheses above must evaluation to a boolean
  # value
  print(paste('val1: ', val1))

  # update condition variable
  val1 = val1 + 1 # IMPORTANT STATEMENT # 2
}
```

```
[1] "val1: 5"
[1] "val1: 6"
[1] "val1: 7"
[1] "val1: 8"
[1] "val1: 9"
```

You can also “drive” the while loop with logical operators.

```
# Using a logical expression
val2 <- 11

while (val2 > 10 && val2 <= 20) {
  print(paste("Value of val2: ", val2))
  val2 <- val2 + 1 # Do you have to only add 1?
}
```

```
[1] "Value of val2: 11"
[1] "Value of val2: 12"
[1] "Value of val2: 13"
[1] "Value of val2: 14"
[1] "Value of val2: 15"
[1] "Value of val2: 16"
[1] "Value of val2: 17"
[1] "Value of val2: 18"
[1] "Value of val2: 19"
[1] "Value of val2: 20"
```

Good code is:

- code that works and
- that can be understood by others.

Efficiency comes with practice and in exploring different ways to write the same code.

Let’s look at some code and see how we can make it more efficient.

```
# Initialize variables
logical.1 <- TRUE
val1 <- 16

while (logical.1 == TRUE) {
  # A wolf leaves
  val1 = val1 - 1

  # the number of wolves drops below five, they get unhappy
  if (val1 <= 5) {
    logical.1 = FALSE
  }
  print(paste('Val1: ', val1))
}
```

```
[1] "Val1: 15"
[1] "Val1: 14"
[1] "Val1: 13"
[1] "Val1: 12"
[1] "Val1: 11"
[1] "Val1: 10"
[1] "Val1: 9"
[1] "Val1: 8"
[1] "Val1: 7"
[1] "Val1: 6"
[1] "Val1: 5"
```

```
# Walk through the steps in your head. Is there anything that seems
# duplicative? Focus on the "if" statement used above to change the
# value of "logical.1".
```

```
# More efficient while loop
```

```
# We use the value of val1 to update the logical variable logical.1
# The variable logical.1 plays no other role, we can therefore use
# val1 directly to control the loop
```

```
val1 <- 16
```

```
# we can directly check number of wolves, but we need to change the
# relational operator (why?)
```

```
while (val1 > 5) {
  # reduce val1 by 1
  val1 = val1 - 1
  # output the value
  print(paste('Val1: ', val1))
}
```

```
[1] "Val1: 15"
[1] "Val1: 14"
[1] "Val1: 13"
[1] "Val1: 12"
[1] "Val1: 11"
[1] "Val1: 10"
[1] "Val1: 9"
[1] "Val1: 8"
[1] "Val1: 7"
[1] "Val1: 6"
```

```
[1] "Val1: 5"
```

```
# check outputs for the inefficient version and this one.  
# Are the outputs the same?
```

Inefficient while loops are likely to occur if you use a logical variable for the while loop test condition. The logical variable will have to be updated using a logical expression (using relational or logical operators). It is usually more efficient to use the logical expression directly than use a logical variable that is updated using the logical expression. However, you may decide to use the inefficient code to maintain readability/interpretability, i.e. make clear to other users what your code is doing.

Some important considerations with while loops:

make sure you initialize condition values BEFORE the while loop starts

make sure values in the test condition are getting updated appropriately within the loop

I digress ...

Sometimes you need to exit an if, for, or while loop before all the code is executed. Most languages, including R, have a reserved word you can use to exit a loop, if needed. This word is `break`. While we won't cover it today, it is an important enough concept that I want to provide an example.

```
# Use a for loop as an example, but this will work with if and while statements  
# too  
  
for (i in 1:10) {  
  print(paste("Current value:", i))  
  
  # if value = 7 print a different message and exit loop  
  if (i == 7) {  
    print("Elvis has left the building. Bye!")  
    break # this statement will end the loop  
    # What would happen if you changed the 7 to 11?  
  }  
}
```

```
[1] "Current value: 1"  
[1] "Current value: 2"  
[1] "Current value: 3"
```

```
[1] "Current value: 4"  
[1] "Current value: 5"  
[1] "Current value: 6"  
[1] "Current value: 7"  
[1] "Elvis has left the building. Bye!"
```